

01 · Descripción general del sistema

Documentacion del sistema

Version analizada: **0.3.0**

Commit analizado: **ff246ab**

Fecha de generacion: **2026-08-27**

Generado a partir de 01-system-overview.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Qué es
2. Qué problema resuelve
3. A quién está dirigido
4. Casos de uso: el catálogo verificado
5. Flujo general
6. Entradas y salidas
7. La cuestión de la IA: qué hay y qué no hay
8. Componentes más importantes
9. Tecnologías y dependencias
10. Límites del sistema
11. Integraciones externas
12. Estado general observado
13. El sistema explicado para una persona no técnica

Qué es Automa, qué problema resuelve, quién lo usa, qué entra y qué sale. Al final hay una sección escrita para alguien sin formación técnica.

1. Qué es

Automa es un orquestador local de automatización (RPA) para el escritorio Windows.

Cada tarea automatizable se describe en un archivo JSON llamado *manifest* (`flows/<carpeta>/manifest.json`). Ese archivo declara una lista de pasos, cada paso invoca una **acción** con nombre (`screen.capture_screenshot`, `browser.fill_form`, `system.run_powershell...`), y opcionalmente declara condiciones, reintentos y transiciones entre pasos. Un motor determinista (`engine/orchestrator.py`) lee el manifest, aplica una política de seguridad, ejecuta los pasos en orden y persiste todo lo que ocurrió.

Tres propiedades definen el producto y conviene protegerlas:

1. **Local-first.** Todo corre en `127.0.0.1`. No hay backend remoto, no hay cuenta, no hay telemetría saliente. La única persistencia es un SQLite en disco.
2. **Determinista y sin LLM.** No hay ningún modelo de lenguaje en el camino de ejecución. La lógica de decisión es `engine/conditions.py` (13 operadores de comparación) y `actions/rules.py` (reglas declarativas). Ver §7.
3. **Declarativo.** Añadir un caso de uso es añadir una carpeta con un JSON. No se toca el motor. Los 27 flows actuales están contruidos con las 36 acciones existentes.

2. Qué problema resuelve

Un operador que trabaja sobre Windows repite tareas que ninguna herramienta única cubre: capturar el escritorio y dejar evidencia fechada, rellenar un formulario web con datos de un dataset, inventariar una carpeta, vigilar si una página cambió, auditar los enlaces de un sitio, tomar un snapshot de recursos del equipo.

Cada una de esas tareas es fácil de scriptear una vez. Lo difícil, y lo que Automa aporta, es lo que viene después:

Problema	Cómo lo resuelve Automa
Cada script vive en su propio archivo con su propia forma	Contrato único: <code>schemas/manifest.schema.json</code> , validado por <code>scripts/validate_project.py</code>
No queda registro de qué se ejecutó, cuándo y con qué resultado	Cada corrida escribe en SQLite (<code>runs</code> , <code>steps</code> , <code>events</code>), en JSONL (<code>logs/</code>) y en un snapshot JSON (<code>state/</code>)
Un script puede escribir donde quiera	<code>SandboxPolicy</code> puede restringir acciones, rutas y tiempo máximo por flow
Programar la tarea implica salir a Task Scheduler	Scheduler propio con intervalo o cron de 5 campos y lock en SQLite
Hay que abrir una terminal para lanzar nada	Panel local de 3 pestañas con atajos <code>Alt+N</code> , o ventana nativa <code>pywebview</code>

3. A quién está dirigido

Actor	Qué hace con el sistema	Evidencia
Operador local	Único rol con acceso. Ejecuta flows desde el panel, programa horarios, consulta el histórico	El panel escucha en <code>127.0.0.1:8787</code> (<code>app/server.py::run_server</code>) y no tiene usuarios ni roles
Autor de flows	Escribe <code>manifest.json</code> nuevos usando las acciones existentes	Contrato en <code>docs/CREAR_FLUJOS.md</code> y <code>schemas/manifest.schema.json</code>

Desarrollador de acciones	Añade funciones a <code>actions/</code> y las registra en <code>_BUILT_IN_ACTIONS</code>	<code>engine/action_registry.py</code>
Integrador externo	Dispara un flow por webhook <code>POST /api/hook/<folder></code>	Requiere <code>AUTOMA_WEBHOOK_TOKEN</code> ; deshabilitado si no está definido
Sistema de monitoreo	Consume <code>GET /metrics</code> en formato Prometheus	<code>engine/metrics.py::prometheus_text</code>

No existe multiusuario, no existe RBAC y no existe sesión de usuario. El propio README del repositorio lo declara: «Multiusuario / RBAC — [alto] No — un operador local».

4. Casos de uso: el catálogo verificado

27 flows en `flows/`, agrupados por la clave `family` de su manifest. Conteo real obtenido leyendo los 27 manifests:

Familia	Flows	Qué caracteriza al grupo
<code>sistema</code>	10	Interactúan con componentes nativos de Windows o leen el estado del equipo
<code>navegador</code>	9	Lanzan Chromium con Playwright: capturan, rellenan o leen el DOM
<code>pantalla</code>	6	Capturan píxeles del escritorio, con o sin OCR posterior
<code>filesystem</code>	1	Inventario de una carpeta
<code>documentos</code>	1	Resumen de archivos de texto de una carpeta

Catálogo completo, con sus pasos y su política de sandbox real, en [04 · Mapa del código](#) y [19 · Matriz de trazabilidad](#).

Cinco ejemplos que ilustran el rango del sistema:

- **07_browser_form_filler** — la operación más compleja del repositorio. Carga 100 registros de `data/seeds/form_seeds.json`, elige uno que no se haya usado antes (tracking persistente en `data/seeds/.used_indices.json`), lanza Chromium **visible** con `slow_mo=250 ms`, rellena 10 campos uno a uno, envía, lee la validación JavaScript de la página y persiste el payload. Un solo paso de manifest; toda la complejidad vive en `actions/browser_form.py::fill_form`.
- **23_web_change_detector** — extrae el texto de una página, calcula su SHA-256, lo compara con la corrida anterior y **solo si cambió** dispara `notify.send`. La condición está en el manifest (`"when": {"path": "decision.status", "operator": "eq", "value": "alerta"}`), no en el código.
- **18_powershell_audit** — ejecuta un comando PowerShell contra una allowlist de 13 verbos de solo lectura y guarda `stdout/stderr/exit_code`.
- **08_windows_lock_workstation** — un paso, `ui.hotkey` con `Win+L`. Con `allowed_actions` de un solo elemento y `max_runtime_seconds: 5`.
- **05_system_healthcheck** — snapshot de CPU/RAM/disco con `psutil`, evaluación por reglas y reporte JSON. Es el flow que usa el smoke test.

5. Flujo general

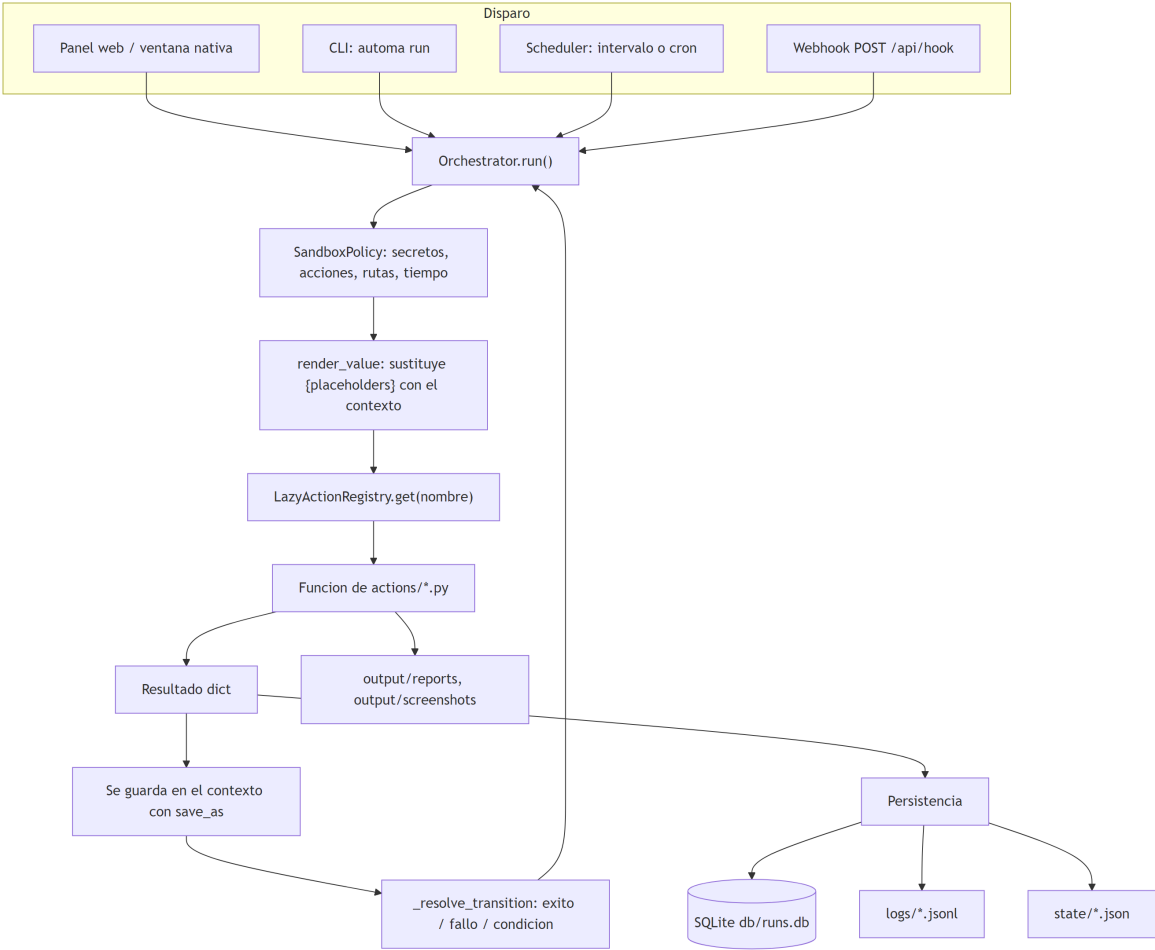


Diagrama 1 (Mermaid, renderizado)

Lo que el diagrama muestra: los cuatro disparadores convergen en el mismo **Orchestrator**, la política se aplica antes de cada acción, y cada paso realimenta el contexto del que se alimenta el siguiente. La persistencia es triple y simultánea.

Lo que el diagrama NO muestra: que el bucle es **síncrono y de un solo hilo dentro de una corrida** — un paso lento bloquea el flow entero, y **max_runtime_seconds** solo se comprueba *entre* pasos, nunca interrumpe uno en curso. Tampoco muestra que el panel lanza la corrida en un hilo aparte (**threading.Thread** en **do_POST**) para devolver el **run_id** de inmediato y dejar que el navegador haga polling.

6. Entradas y salidas

Entradas

Entrada	Origen	Módulo que la lee
manifest.json	flows/<carpeta>/	engine/loader.py::FlowLoader.load_manifest
Contexto del flow	configs/<carpeta>.json → flows/<c>/context.user.json → flows/<c>/context.example.json, en ese orden de prioridad	engine/loader.py::FlowLoader.load_context
context_overrides	Body JSON de POST /api/run/<folder>	app/server.py::do_POST → Orchestrator.__init__
Secretos	Variables de entorno, con respaldo en secrets/secrets.json	engine/secrets.py::get_secret
Estado del equipo	psutil, mss, pyperclip, PowerShell	actions/system.py, actions/screen.py

Páginas web	Chromium vía Playwright	<code>actions/browser_*.py</code>
-------------	-------------------------	-----------------------------------

Salidas

Salida	Ruta	Escrita por
Historial estructurado	<code>db/runs.db</code> (7 tablas)	<code>engine/database.py</code>
Log de eventos por corrida	<code>logs/<flow_id>_<run_id>.jsonl</code>	<code>engine/logger.py::JsonLLogger</code>
Snapshot completo del estado	<code>state/<flow_id>_<run_id>.json</code>	<code>engine/state_store.py::StateStore</code>
Reportes de los flows	<code>output/reports/*.json, *.csv, *.md</code>	<code>actions/filesystem.py::write_json,</code> <code>actions/browser_extract.py</code>
Capturas	<code>output/screenshots/*.png</code>	<code>actions/screen.py,</code> <code>actions/browser_capture.py</code>
Métricas	<code>GET /metrics</code> (texto Prometheus)	<code>engine/metrics.py::prometheus_text</code>

Detalle no obvio: `engine/introspection.py::extract_existing_paths` decide qué cuenta como "output" de una corrida recorriendo el estado en busca de rutas de archivo existentes, pero **solo acepta las que están bajo `output/`**. Un comentario del propio archivo explica el porqué: antes contaba cualquier ruta existente y contaminaba la lista con los archivos que el flow solo había *leído*.

7. La cuestión de la IA: qué hay y qué no hay

El repositorio se presenta explícitamente como «sin IA». La verificación en el código confirma esa afirmación para el catálogo publicado, con un matiz que conviene registrar:

Hecho	Evidencia
Ningún flow del catálogo llama a un modelo de lenguaje	Las 26 acciones distintas usadas por los 27 manifests son de filesystem, pantalla, sistema, UI, reglas, HTTP, notificación y navegador
La toma de decisiones es determinista	<code>engine/conditions.py::matches</code> implementa 13 operadores (<code>eq</code> , <code>gt</code> , <code>contains</code> , <code>regex</code> ...). <code>actions/rules.py::evaluate_rules</code> evalúa reglas declaradas en el manifest y devuelve la primera que coincide
Existe un adaptador de visión multimodal	<code>plugins/analyzers/vision_model_analyzer.py::VisionModelAnalyzer</code> soporta <code>mock</code> , <code>openai_compatible</code> y <code>ollama</code>
Ese adaptador no es alcanzable desde ningún flow	Solo lo usa <code>actions/vision.py::inspect_screen_target</code> , y esa acción no aparece en ninguno de los 27 manifests. Verificado programáticamente
Su modo por defecto no llama a ninguna red	<code>provider='mock'</code> produce una lectura heurística local de brillo y RGB con Pillow
<code>decision/optional_ai.py</code> es un stub	<code>suggest_step_order</code> devuelve la lista sin tocarla y nadie lo importa . Ver 15

Conclusión: el sistema es determinista de extremo a extremo tal y como se distribuye. El adaptador de visión es un punto de extensión latente, sin uso, cuya activación exigiría escribir un flow nuevo que invoque `vision.inspect_screen_target` con un `vision_provider` distinto de `mock`. Documentado en detalle en 09 · APIs e integraciones.

8. Componentes más importantes

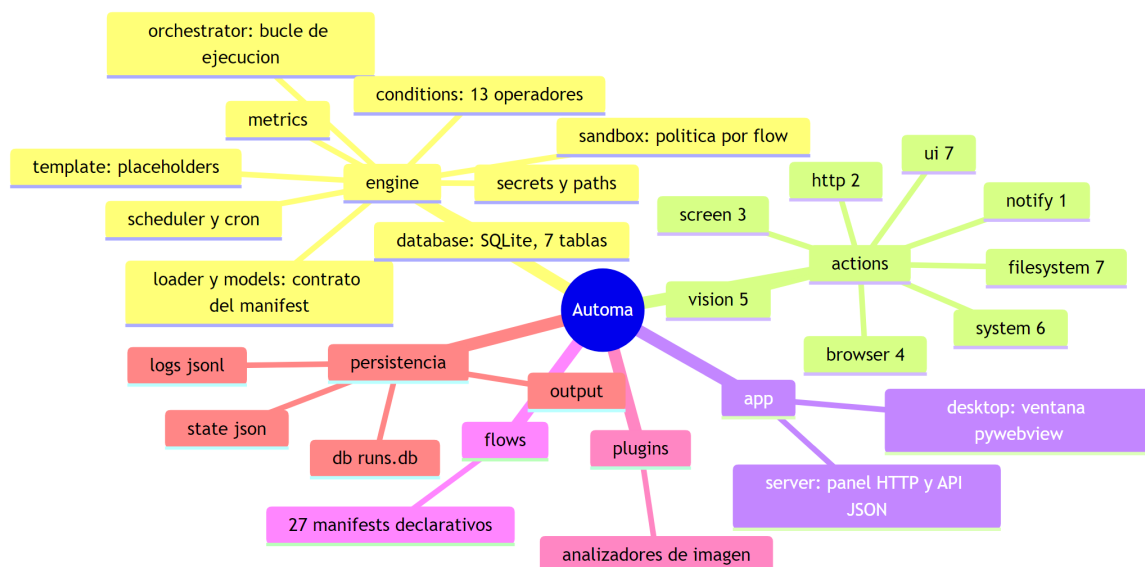


Diagrama 2 (Mermaid, renderizado)

Lo que el mapa muestra: las cinco piezas del sistema y el reparto de las 36 acciones por familia. **Lo que no muestra:** que `app/server.py` concentra 1 753 líneas —el 29 % del Python de producción— porque además del enrutado HTTP contiene el HTML, el CSS y el JavaScript del panel embebidos como cadenas Python. Es el archivo con más peso y el que más cuidado exige al modificar (ver 15).

9. Tecnologías y dependencias

Dependencias declaradas en `pyproject.toml`

Paquete	Cota	Para qué se usa
<code>Pillow</code>	<code>>=12.2.0, <13</code>	Análisis de imagen, recorte, captura de respaldo
<code>psutil</code>	<code>>=5.9.8, <8</code>	CPU, memoria, disco, procesos
<code>requests</code>	<code>>=2.32.2, <3</code>	<code>http.fetch_url</code> , <code>http.check_urls</code> , webhooks de salida, <code>robots.txt</code>
<code>mss</code>	<code>>=9.0.1, <11</code>	Captura de pantalla primaria
<code>pyautogui</code>	<code>>=0.9.54, <1</code>	Teclado y ratón
<code>pytesseract</code>	<code>>=0.3.10, <1</code>	Puente al binario OCR <code>tesseract</code>
<code>pyperclip</code>	<code>>=1.8.2, <2</code>	Lectura del portapapeles
<code>PyGetWindow</code>	<code>>=0.0.9, <1</code>	Rectángulo de la ventana activa
<code>pywebview</code>	<code>>=5.4, <7</code>	Ventana nativa del escritorio

Extras: `schema` (`jsonschema>=4`) y `dev` (`pytest`, `pytest-cov`, `ruff`, `jsonschema`, `pre-commit`).

Un comentario del propio `pyproject.toml` explica la política de cotas: «Cotas de seguridad: piso explícito = primera versión sin CVE conocida al corte de la auditoría 2026-06-01». Es una decisión documentada, no un rango arbitrario.

Dependencia crítica que NO está declarada

`playwright` **no aparece** en `pyproject.toml` ni en `requirements.txt`, pero nueve flows (02, 07, 21–27) no funcionan sin él. Las tres acciones que lo usan lo importan dentro de la función y levantan un `RuntimeError` con la instrucción de instalación:

```
# actions/browser_capture.py – patrón repetido en browser_form.py y browser_extract.py
raise RuntimeError(
    'playwright no está instalado. Ejecuta: '
    'pip install playwright && python -m playwright install chromium'
) from exc
```

Es una degradación explícita y con mensaje útil, no un crash opaco. Aun así, un tercio del catálogo depende de un paquete que `pip install -e .` no instala. Registrado en [15 · Riesgos y deuda técnica](#).

Dependencia externa opcional: `tesseract`

`vision.ocr_image` necesita el binario `tesseract` instalado en el sistema. `plugins/analyzers/ocr_image_analyzer.py::OCRImageAnalyzer` lo comprueba antes de usarlo y, si falta, devuelve `status: "unavailable"` con instrucciones por sistema operativo en vez de fallar. Un comentario del archivo explica la decisión: permite que un flow con rama de recuperación siga su camino alternativo.

10. Límites del sistema

Lo que Automa **no** hace, y conviene decirlo antes de que alguien lo suponga:

- **No aísla a nivel de sistema operativo.** El sandbox es declarativo: el motor comprueba nombres de acción y prefijos de ruta antes de llamar a la función. Una vez dentro, la acción corre con todos los permisos del usuario. [docs/SEGURIDAD.md](#) lo declara y el README lo marca como «[bajo] Sandbox declarativo, no proceso».
- **Ese sandbox es opcional y 14 de 27 flows no lo usan.** Solo los flows 08–20 declaran `allowed_actions`; los flows 01–07 y 21–27 corren con `SandboxPolicy` completamente permisiva. Detalle y conteo en [11 · Seguridad](#).
- **No hay autenticación de usuario.** Hay protección anti-CSRF y un token opcional (`AUTOMA_PANEL_TOKEN`), pero cualquiera con acceso a la sesión de Windows tiene acceso total al panel.
- **No hay retención ni rotación.** `db/runs.db`, `logs/*.jsonl` y `state/*.json` crecen sin límite. **NO IDENTIFICADO:** no existe ninguna rutina de purga en el repositorio.
- **No hay multiusuario ni permisos.**
- **Los atajos de teclado del panel cubren 12 flows de 27.** El propio README lo advierte: los casos por encima del 12 se ejecutan haciendo clic en su tarjeta.
- **Windows es la plataforma real.** La CI corre también en Ubuntu, pero eso valida la lógica pura y el motor; las acciones de UI, portapapeles, PowerShell y ventana activa no tienen sentido fuera de Windows.

11. Integraciones externas

Integración	Dirección	Autenticación	Módulo
Webhook entrante	Entra	<code>AUTOMA_WEBHOOK_TOKEN</code> en el header <code>X-Automa-Token</code>	<code>app/server.py::_check_webhook_token</code>
Notificación por webhook	Sale	<code>Bearer</code> opcional, con soporte <code>@secret:NOMBRE</code>	<code>actions/notify.py::send_notification</code>
Scraping / captura web	Sale	Ninguna	Playwright + Chromium
Verificación de enlaces	Sale	Ninguna	<code>actions/http_actions.py::check_urls</code>
<code>robots.txt</code>	Sale	Ninguna	<code>actions/browser_extract.py::RobotsCache</code>
Prometheus	Entra (lectura)	Ninguna	<code>GET /metrics</code>
Proveedor de visión	Sale	<code>Bearer</code> opcional	<code>VisionModelAnalyzer</code> , sin uso

Todas las salidas de red son **opt-in por manifest**: si el flow no declara un paso que las use, no hay tráfico. Los siete flows web (21–27) apuntan por defecto a HTML locales del propio repositorio (`data/web/`), de modo que la demo funciona sin internet.

12. Estado general observado

Medido en el commit analizado, ejecutando los comandos del propio repositorio:

Señal	Resultado	Comando
Suite de pruebas	150 pasan, 0 fallan en 16,62 s	<code>python -m pytest</code>
Cobertura	58,92 %, umbral configurado 54 %	incluida en <code>pytest</code> vía <code>--cov-fail-under=54</code>
Lint	Sin hallazgos	<code>python -m ruff check .</code>
Validación de manifests	27 flows, 36 acciones, 0 errores	<code>python scripts/validate_project.py</code>
Árbol de trabajo	Limpio al iniciar el análisis	<code>git status --porcelain</code>

La CI tiene cinco workflows (`ci`, `security`, `workflow-security`, `markdown-docs`, `dependency-hygiene`) más el de release. Todas las acciones de terceros están fijadas a SHA, hay un verificador propio (`pin-check`) que falla si alguien introduce un `uses:` sin SHA, y `security.yml` corre CodeQL `security-extended`, `detect-secrets` sobre el filesystem y los últimos 50 commits, y `pip-audit`.

Lectura honesta del estado: el sustrato es sólido y está bien probado en su lógica pura. Los huecos están donde el sistema toca el mundo real: las acciones de UI, captura y navegador tienen poca o ninguna cobertura automatizada, y el empaquetado tiene un hallazgo abierto (§10 del 15).

13. El sistema explicado para una persona no técnica

Imagine que tiene un ayudante muy obediente y muy literal sentado frente a su computador.

Usted le deja una **receta escrita**: «primero saca una foto de la pantalla, después guarda un informe con lo que veas, y si el informe dice que la memoria está por encima del 80 %, avísame». El ayudante hace exactamente eso, ni más ni menos, en ese orden. No improvisa, no interpreta, no consulta a nadie. Si un paso falla, mira la receta a ver si usted dejó escrito qué hacer en ese caso.

Eso es Automa. Las recetas son archivos de texto que se pueden leer y corregir; el ayudante es un programa que vive en su computador y no habla con internet salvo que la receta se lo pida.

Lo que gana con esto:

- **Queda registro de todo.** Cada vez que el ayudante ejecuta una receta anota en un cuaderno la hora, cada paso, cuánto tardó y qué resultado dio. Puede volver a mirarlo meses después.
- **Las recetas se pueden acotar.** Puede escribir en la receta «solo puedes escribir en esta carpeta» o «si tardas más de cinco segundos, para». El programa lo respeta.
- **Puede programarlo.** «Ejecuta esta receta cada quince minutos» o «todos los lunes a las nueve».
- **Todo se maneja desde una ventana.** Hay una pantalla con tarjetas: una por receta. Se hace clic y el ayudante empieza, mostrando en qué paso va.

Lo que NO hace, y es importante saberlo antes de confiar en él:

- **No piensa.** No hay inteligencia artificial decidiendo nada. Si la receta está mal escrita, hará lo que dice la receta, no lo que usted quería decir.
- **No es una caja fuerte.** Las restricciones de la receta las hace cumplir el propio programa, no el sistema operativo. Es como un cartel de «no pasar»: funciona si quien pasa respeta el cartel. Y además, catorce de las veintisiete recetas actuales no llevan cartel puesto.
- **Cualquiera que use su computador puede usarlo.** No hay contraseña de usuario. Si alguien se sienta en su sesión de Windows abierta, puede ejecutar cualquier receta.
- **No borra nada solo.** El cuaderno de registro y las capturas se acumulan indefinidamente. Con el tiempo ocupan espacio y hay que limpiarlos a mano.

- **Es para Windows.** Buena parte de las recetas (abrir el Explorador, bloquear la sesión, leer el portapapeles) solo tienen sentido ahí.
- **Algunas recetas necesitan piezas extra.** Las que abren páginas web necesitan que antes se instale un navegador especial; las que leen texto de una imagen necesitan un programa de reconocimiento de texto. Si falta la pieza, el programa lo dice con claridad en vez de fallar de forma confusa.

En una frase: **Automa convierte tareas repetitivas de Windows en recetas escritas que se ejecutan solas, dejando rastro de todo, sin enviar nada a internet y sin adivinar nada.**

Documentos relacionados: [02 · Instalación](#) · [03 · Arquitectura](#) · [11 · Seguridad](#) · [15 · Riesgos](#) · [17 · Resumen ejecutivo](#)